Microsoft Azure

Microsoft Foundry

From prototype to operated service

Build with GitHub Copilot.

Run, evaluate and improve in Foundry.

Sridhar Arrabelly

Customer deep dive + live demo | 50 minutes + Q&A

One customer request. A complete operating story.

Build quickly. Prove deliberately. Learn after release.



Build	Connect	Prove	Release	Improve
Copilot + code	Business tools	Cases + traces	Version + evidence	Observed failure
A useful assistant	Checkable facts	Measured behaviour	An explicit gate	A reviewed new test

The strength is in the connections—not the number of features we can show.

One use case throughout

Meet Volt & Co

A fictional electronics assistant that recommends the cheapest product that works.

“A charger for my Volt Book 14. Under EUR 50.”

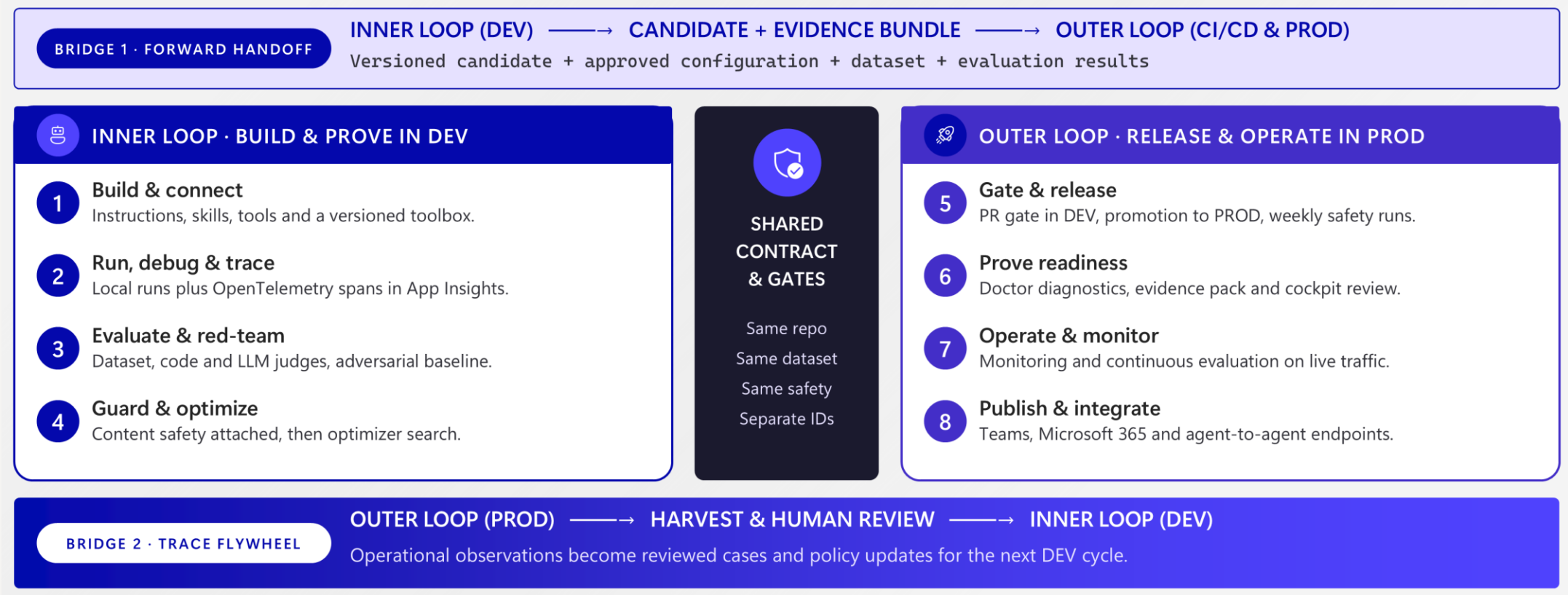
The device requires 65W.

Volt Charge 30	30W | EUR 19.90	**Underpowered**
Volt Charge 65	65W | EUR 39.90	**Compatible + in budget**
Volt Charge 100	100W | EUR 59.90	**Above budget**

Then the same customer asks about a recall. Missing evidence becomes the next test.

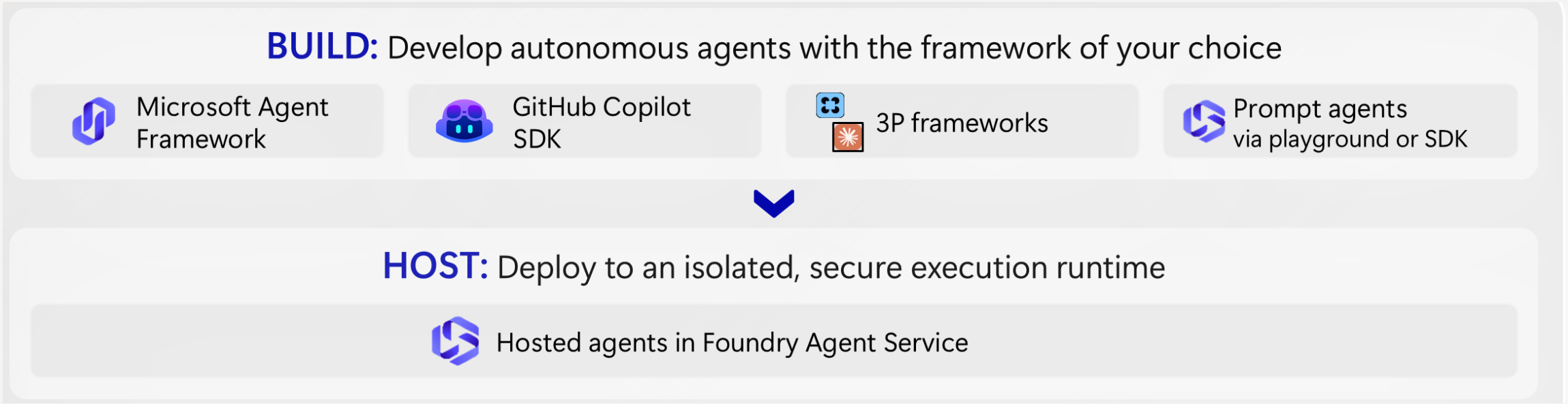
One continuous system: inner and outer loop

Reference lifecycle: two loops, connected by versioned and reviewed evidence.



Build in GitHub. Run and improve in Foundry.

Copilot for development. Copilot SDK for runtime. Foundry for hosting and lifecycle.



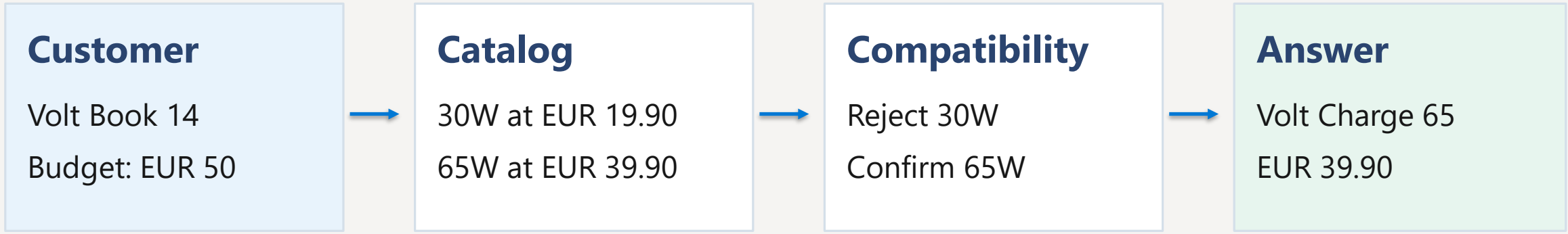
Inspect every run **Score before release** **Learn from traffic**

Live demo 1

Watch the answer—and the path that produced it

One request, three business checks, inspectable evidence.

LIVE DEMO 01 09:00-16:00 | live request + trace



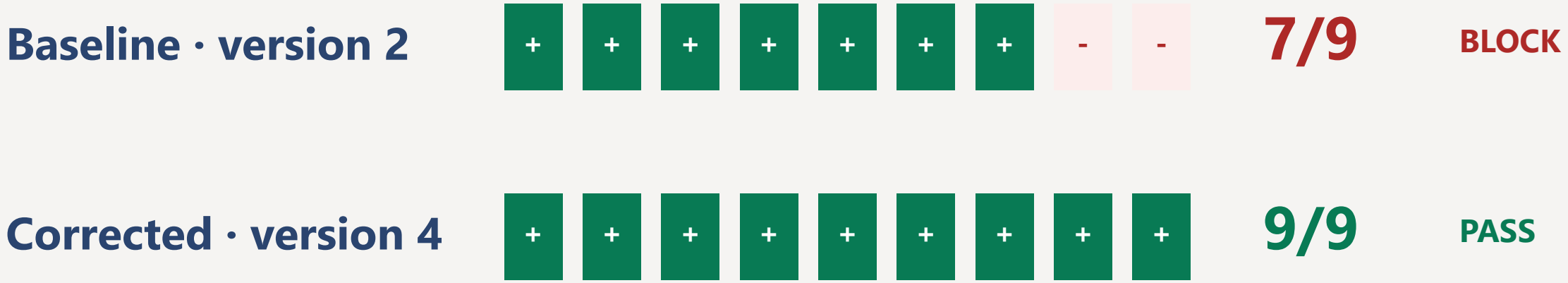
The trace links the request, tool arguments, returned facts and response.

Next: explaining one execution is not the same as proving consistent behaviour.

Evaluate

From a convincing answer to measurable evidence

Recorded rehearsal: the same nine synthetic cases, compared across two versions.



The two failed cases both omitted the batch code.

Small synthetic-case results—not a population accuracy estimate or safety certification.

Three checks. Three different jobs.

Use deterministic checks, model judges and human review together.

Code-based

Objective business assertions

Correct product and batch

Required tool evidence

Missing-input contract

Model-based

Native Foundry scorecard

Coherence of captured answers

A semantic quality signal

Not domain correctness

Human

Expected behaviour and acceptance

Ambiguous or risky cases

Validate the labels

Approve the release policy

No single score can establish usefulness, correctness and safety for every case.

Live demo 2

Change the contract—not just the wording

“I have a Volt Charge 65. Is it safe?”

LIVE DEMO 02 18:00-25:00 | baseline, correction, comparison

BEFORE

```
batch = null
status = no_recall
```

AFTER

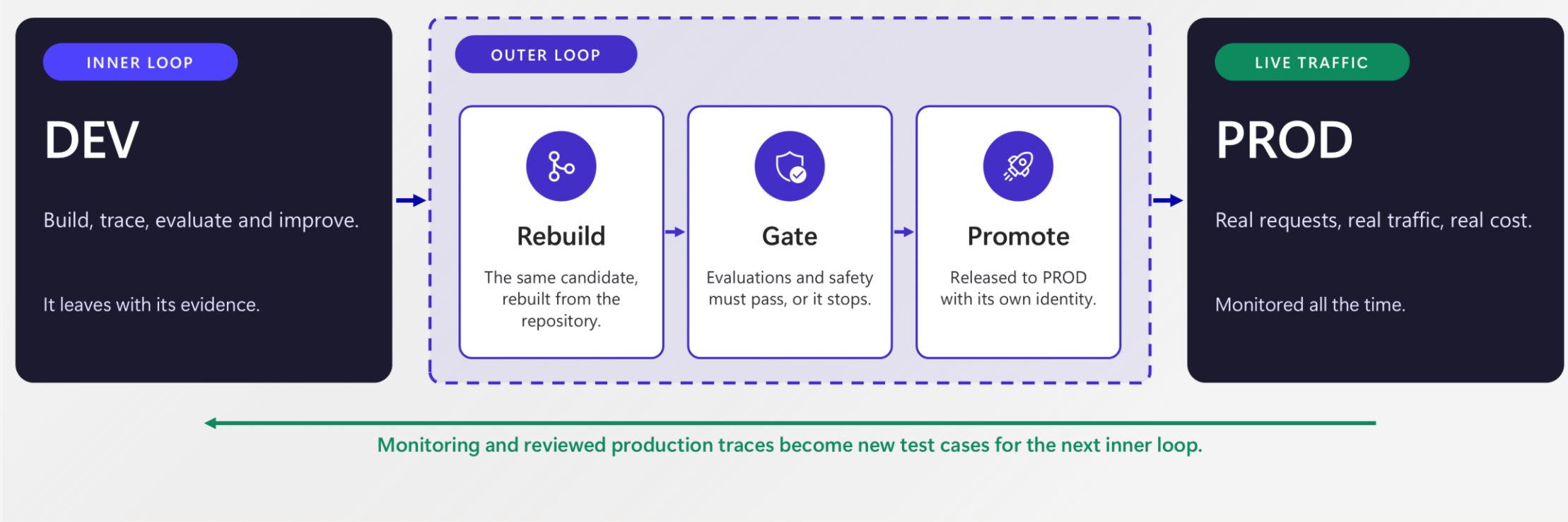
```
batch = null
status = needs_batch
```

Missing evidence was treated as a negative lookup.

Ask for the batch before making a recall conclusion.

How a candidate becomes an operated service

Reference pattern: rebuild, gate and promote—then learn from operation.



Our demo: local DEV + a hosted DEMO target. The approved client pins the evaluated version.

Live demo 3

A release we can explain

Show the actual Actions run and the evidence attached to the candidate.

LIVE DEMO 03 27:00-34:00 | gate + deployed version

1 → 2 → 3 → 4

Source

Git revision

Configuration

Evidence

Dataset hashes

Case results

Decision

9/9 criteria pass

Explicit acceptance

Serving

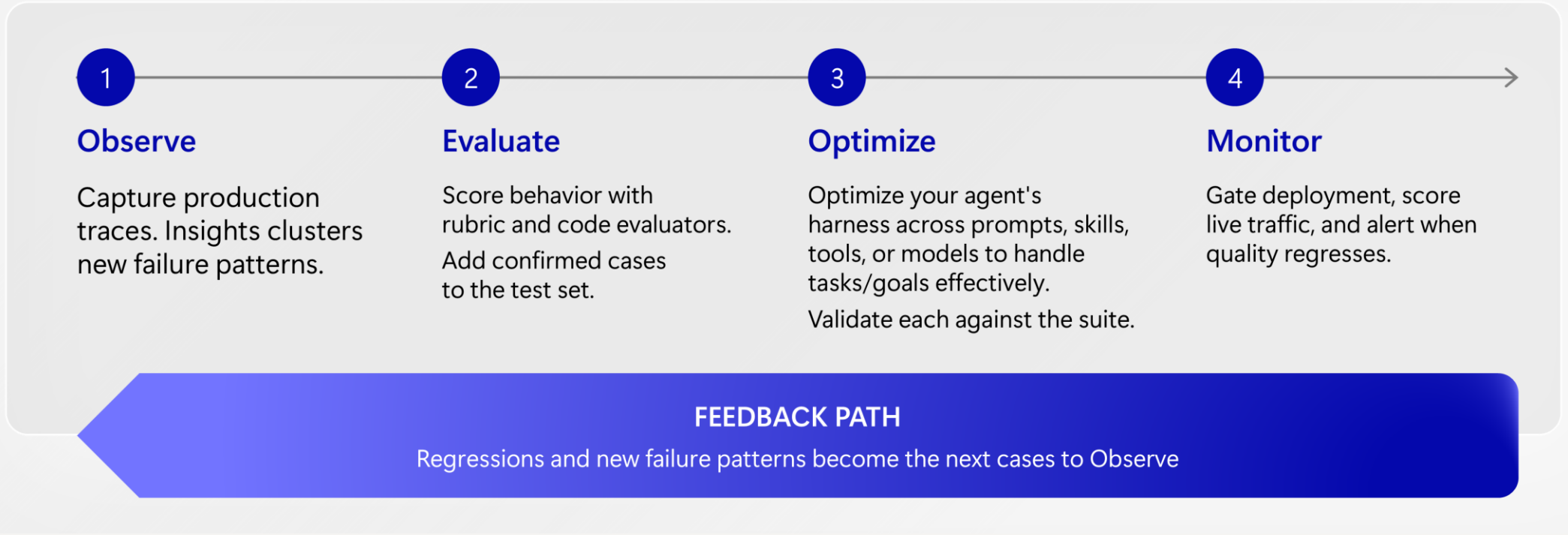
volt-assistant:4

Pinned demo client

Deployment and long-running scoring were completed in rehearsal—not silently simulated.

Operations teaches development what to test

The next learning cycle starts with evidence from a real interaction.



In this session: a synthetic hosted-agent request becomes a reviewed regression case.

Live demo 4

Close the loop with one new test

Observed output is evidence. It is not automatically a correct label.

LIVE DEMO 04

35:00-47:00 | trace to reviewed case to corrected candidate

Observe

No batch supplied.

Baseline returned no_recall.

Review

Expected: ask for the batch.

Keep the source trace ID.

Prove

Run the new regression.

Corrected version passes.

Return to DEV with a stronger contract—not a direct patch to the serving prompt.

Microsoft Azure

One system. Four things to remember.

1. Start with a customer outcome.
2. Prove the path, not just the prose.
3. Make the release decision explicit.
4. Turn observations into reviewed tests.

Microsoft Azure

Thank you

Which customer journey would you put through this loop?

Outcome · Boundaries · Ownership

Sridhar Arrabelly

Q&A | 50:00-60:00